

27/5/26

Assignment

1) What is the output of the following, and why?

`s = "Python"`

`print(s[0], s[-1], s[6])`

Soln:- Error, Because there is no `s[6]`, 6 index value and the negative value should come last.

2) What is the difference between positive and negative indexing?

Given `s = "Hello"`, what does `s[-2]` return?

Soln:- positive $\rightarrow$ traversing Negative index $\rightarrow$ back traversing/
back tracking
`s[-2]  $\Rightarrow$  l`

3) What happens when you access an index that is out of range in a string? How is this different from slicing out of range?

`s = "code"`
0 1 2 3

Soln:- Here the index value is 0-3 so it is out of range. When we access this, error will come.

`print(s[10])` #vs

`print(s[1:10])`

`print(s[10])  $\Rightarrow$  Indexing  $\Rightarrow$  Error because no value exist`

`print(s[1:10])  $\Rightarrow$  slicing  $\Rightarrow$  1, 2, 3.`

4) How do you find the index of the first and last occurrence of a character in a string using indexing concepts?

`s = "mississippi"` # first index of first 's' and last 's'
0 1 2 3 4 5 6 7 8 9 10

Soln:- first occurrence of a character $\Rightarrow$ we have to find from left side first character `[s]`

last occurrence of a character $\Rightarrow$ we have to find from right side first character `[s]`

first `[s]  $\Rightarrow$  2` last `[s]  $\Rightarrow$  6`

5) What is the output of the following, and explain how negative indexing resolves?

$s = \text{"abcdef"}$, $\text{print}(s[-1], s[-3], s[-5])$
-6 -5 -4 -3 -2 -1

Soln: $s[-1] = f$, $s[-3] = d$, $s[-5] = a$

negative indexing is back traversing / back tracking method

6) What is the syntax for string slicing in python?

Explain each part of $s[\text{start}:\text{stop}:\text{step}]$

Soln: Start $\Rightarrow$ It means starting value should be smaller index value.

Stop $\Rightarrow$ It means stopping value should be greater than index value and it will minus the given index value.

Step $\Rightarrow$ In positive slicing means step increment, Negative slicing means step decrement.

7) What is the output of the following slices and why?

$s = \text{"HelloWorld"}$
0 1 2 3 4 5 6 7 8 9

i) $\text{print}(s[0:5])$ ii) $\text{print}(s[5:])$ iii) $\text{print}(s[:5])$ iv) $\text{print}(s[:])$

Soln: i) Hello $\Rightarrow \text{print}(s[0:5])$

ii) World $\Rightarrow \text{print}(s[5:])$

iii) Hello $\Rightarrow \text{print}(s[:5])$

iv) HelloWorld $\Rightarrow \text{print}(s[:])$

8) What happens when start is greater than stop in a slice with a positive step? What is the output

$s = \text{"python"}$, $\text{print}(s[4:1])$
0 1 2 3 4 5

Soln: Error

9) How do you extract the last 3 characters of a string using slicing, without knowing its length?
Soln: Use negative slicing `[-3:]` this will slice the last 3 characters.

10) What is the difference between these two slices? what does each return? `s = "abcdefgh"` `print(s[:1])` `print(s[:2])`

Soln: `print(s[1::2])`
`print(s[:1])` = a
`print(s[:2])` = ab
`print(s[1::2])` = bdfh

`1::2` ⇒ Every character will print

`1:2` ⇒ Every 2nd character from index 0 will print

`1::2` ⇒ Every 2nd character from index 1 will print.

11) How do you reverse a string using slicing? what is the step value that achieves this?

`s = "python"`

Reverse it using slicing.

Soln: `[::-1]`

no-h-ty

12) What is the output of the following and explain how the negative step works? `s = "abcdefgh"` `print(s[::-1])`

`print(s[-2])` `print(s[6:1-1])`

Soln: `print(s[::-1])` ⇒ hgfedcba

`print(s[-2])` ⇒ h

`print(s[6:1-1])` ⇒ gfedc

13) How do you extract every 3rd character from a string starting from index 1? $S = "a1b2c3d4e5"$

Expected output: "12345".

Soln: `print(s[1::2])`

14) What is the output of the following negative step slices? Explain: start and stop behaviour when step is negative.

$S = "python"$
 0 1 2 3 4 5
i) `print(s[5:0:-1])` ii) `print(s[5::-1])`

ii) `print(s[:0:-1])`

Soln: i) nohty
ii) nohtyp
iii) -

15) How do slices handle start and stop values that exceed string boundaries with a negative step?

$S = "hello"$ `print(s[100::-1])` `print(s[::-100])`

16) How do you extract the middle portion of a string dynamically, regardless of its length?

$S = "abcdefghij"$ # 10 chars # Extract characters from index

2 to second last (exclusive)

Soln: `[2:9]`

17) How do you use slicing to check if a string is a palindrome in a single line?

$S = "racecar"$

Check palindrome using slicing.

soln:

$[s :: -1]$

Palindrome $\Rightarrow$ True.

18) What is the output of the following, and why does python not raise an error?

$s = \text{"Hello"}_{01234}$; `print (s[1:100]); print (s[100:2]); print (s[100:200])`

soln: `print (s[1:100]) = ello`

`print (s[100:2]) =`

`print (s[100:200]) = " "`

Because it takes only exist index value so it is not raise an error.

19) How do you replace a portion of a string using slicing (since strings are immutable)?

$s = \text{"HelloWorld"}$

Replace "World" with "python" using slicing and concatenation.

soln: `Print ('HelloWorld' + s)`

$s = \text{Helloworld}$

$\text{World} = \text{python}$; # $s = \text{python}$

`Print ('Hello' + s)`

Hellopython

20) How does python's slice() object work, and how can you use it as a reusable slicer?

```
s = "abcdefghi"  
    0123456789
```

```
my_slice = slice(1, 8, 2)
```

```
Print (s[my_slice])
```

What is the output, and how can this be reused on other strings?

Soln:

```
s[b, i, c]
```